Files

A file is the fundamental unit of data storage in a file system. It represents a collection of related information that can be textual, binary, executable, or multimedia. Files serve as containers for user data and program instructions, allowing the system to differentiate between distinct types of content.

Every file is identified by a name and an extension. The extension often denotes the file's format or intended usage (e.g., .txt, .exe, .jpg).

Key properties of files include:

- Name: A human-readable identifier.
- Type: Defines the file's purpose or format.
- Size: Indicates the amount of data stored.
- Permissions: Define which users or processes can read, write, or execute the file.

Example:

/home/user/documents/report.txt

Here, report.txt is a text file located in the documents directory under the user home folder.

Directories

A directory (also called a folder) is a special type of file that organizes other files and directories in a hierarchical manner. This structure enables logical grouping of related data and simplifies navigation within the storage system.

Directories can contain:

- Files (data or program files).
- Subdirectories (nested folders).

The structure typically forms a tree hierarchy starting from the root directory (/).

Example of Hierarchical Structure:

```
/
|-- home/
|   |-- user/
|       |-- documents/
|           |-- report.txt
|           |-- notes.docx
|-- etc/
|   |-- config/
```

This organization allows efficient path resolution and intuitive access patterns.

File Organization Techniques

Efficient file organization ensures optimal use of storage and quick data retrieval. Different techniques are employed depending on the use case and system design goals.

1. Sequential Organization

In sequential file organization, data records are stored in a linear order, one after another. This structure is efficient for batch processing and sequential access patterns such as reading log files or processing large datasets.

Advantages:

- Simple to implement.
- Fast for sequential reads and writes.

Disadvantages:

- Inefficient for random access operations.
- Insertion or deletion requires shifting records.

Example:

Block 1 → Record A

Block 2 → Record B

Block 3 → Record C

Accessing

Record C
requires reading all preceding blocks.

---

2. Direct (Hashed) Organization
Direct file organization uses a hashing function to compute the storage location of each record. This allows direct access to data without sequential traversal.
Formula: Storage Address = Hash(Key)
Advantages:

- Fast random access.
- Ideal for real-time applications such as databases.

Disadvantages:

- Collisions may occur if two keys map to the same address.
- Requires additional mechanisms for handling collisions.

Example: If Hash(ID) maps record R101 to block 7 , the file system directly retrieves block 7 for that record.

---

3. Indexed Organization
In indexed file organization, an index is maintained that stores pointers to the physical location of data records. It is similar to an index in a book, allowing both sequential and random access.
Structure Example:
Index Table:
Key | Block Address

A | 5
B | 12
C | 20
Advantages:

- Supports both sequential and random access.
- Faster search and retrieval compared to purely sequential storage.

Disadvantages:

- Requires extra space to store index tables.
- Additional overhead for index maintenance.

This approach is widely used in modern file systems and databases for optimized access performance.

Disk Organization (Short Answer)
Definition
Disk organization refers to the arrangement of data on a disk for efficient storage and retrieval.

Components of Disk Organization
1. Platter
Circular magnetic disk where data is stored.
2. Track
Circular path on platter surface.
3. Sector
Smallest storage unit of a disk.
4. Cylinder
Collection of tracks of same radius on different platters.
5. Read/Write Head
Reads and writes data on disk.

Disk Access Time
Total time required to access data from disk.

$$\text{Access Time} = \text{Seek Time} + \text{Rotational Latency} + \text{Transfer Time}$$

$$\text{Access Time} = \text{Seek Time} + \text{Rotational Latency} + \text{Transfer Time}$$

Types of Formatting
1. Low-Level Formatting
Creates tracks and sectors.
2. High-Level Formatting
Creates file system.

Advantages
- Efficient data storage
- Fast data access
- Better space management

Tape Organization (Short Note)
Definition
Tape organization refers to the method of storing data on magnetic tape, which is a secondary storage device used mainly for backup and archival purposes.
Features of Tape Organization
- Data is stored sequentially.
- Access method is sequential access.
- Data is recorded on magnetic tape reels.

Components
1. Magnetic Tape
Long plastic strip coated with magnetic material.
2. Records
Data stored in the form of records.
3. Blocks
Group of records stored together.
Characteristics
- Sequential data access
- Large storage capacity
- Low cost
- Slow access speed

Advantages
- Economical for backup storage
- High storage capacity
- Reliable for long-term storage

Disadvantages
- Slow data retrieval
- Not suitable for random access
- Sequential processing only

Applications
- Data backup
- Archival systems
- Large data storage

Allocation Strategies: Contiguous, Linked, and Indexed

The method by which a file's data blocks are stored on disk is called **file allocation**. An efficient allocation scheme ensures minimal overhead, quick access, and low fragmentation. There are three primary strategies: **contiguous**, **linked**, and **indexed allocation**. Each offers trade-offs in terms of access time, complexity, and flexibility.
Contiguous Allocation
Definition: Contiguous allocation assigns a sequence of adjacent disk blocks to each file. This ensures that all parts of a file are stored together, providing fast sequential access.
Illustration:
File A → [Block 1][Block 2][Block 3]

File B → [Block 4][Block 5][Block 6][Block 7]
In this example, File A occupies three contiguous blocks, followed by File B which occupies four. The file system maintains two values for each file:

- Starting block address
- File length (in blocks)

Advantages:

- Fast sequential and random access (only the starting block and length are needed).
- Minimal overhead during file access.
- Simple to implement in both hardware and software.

Disadvantages:

- **External fragmentation:** Free spaces may become scattered over time.
- Difficulty expanding files dynamically; may require moving them to new contiguous areas.
- Inefficient for systems where file sizes change frequently.

Use Case: Commonly used in older file systems and read-heavy workloads where files remain static (e.g., CD-ROM file systems).

Linked Allocation
Definition: In linked allocation, each file is stored as a chain of disk blocks, where each block contains a pointer to the next. The blocks need not be contiguous, thus reducing external fragmentation.
Illustration:
File A: [Block 3 | → 7] → [Block 7 | → 15] → [Block 15 | → NULL]
Here, the file starts at block 3, continues to block 7, and ends at block 15.
Advantages:

- Eliminates external fragmentation.
- Flexible in accommodating file growth.
- Easy insertion and deletion of blocks.

Disadvantages:

- Slower random access due to traversal through linked pointers.
- Overhead of storing pointers reduces usable data space.
- Prone to data loss if a block pointer becomes corrupted.

Use Case: Used in sequential-access file systems or removable storage media where file expansion is frequent.

Indexed Allocation
Definition: Indexed allocation uses an **index block** to maintain pointers to all the data blocks of a file. Each file has its own index block, acting like a table of contents for that file's data.
Illustration:
Index Block of File A: [5, 9, 11, 12, 14]
File A → [Block 5][Block 9][Block 11][Block 12][Block 14]
The index block contains the addresses of all blocks that store parts of File A.
Advantages:

- Supports direct access to any block.
- Avoids external fragmentation.
- Efficient for large files as additional index levels can be introduced (multi-level indexing).

Disadvantages:

- Requires extra space for the index block.
- Slightly higher overhead during small file access due to the need to read the index block first.

Use Case: Widely used in modern file systems like UNIX (using inodes) and NTFS (using Master File Table).
System Calls for File Management
Definition
File management system calls are system calls used by the Operating System to perform operations on files such as creating, reading, writing, and deleting files.
These system calls provide an interface between user programs and the OS.

Common File Management System Calls
1. Create File

Used to create a new file.
Example
create("file.txt");
Function

- Allocates space for file
- Creates file entry in directory

---

2. Open File
Used to open an existing file.
Example
open("file.txt");
Function

- Checks file existence
- Loads file information into memory

---

3. Read File
Reads data from a file.
Example
read(fd, buffer, size);
Function

- Transfers data from file to memory

---

4. Write File
Writes data into a file.
Example
write(fd, buffer, size);
Function

- Stores data from memory to file

---

5. Close File
Closes an opened file.
Example
close(fd);
Function

- Releases file resources

---

6. Delete File
Removes a file from storage.
Example
delete("file.txt");
Function

- Removes file entry and frees space